Flex OPF

Andrea Bragantini

December 2021

Introduction

This paper presents a version of the AC optimal power flow optimization problem used for the activation of flexibility, in particular for the calculation of the flexibility request to be than offered in an hypothetical flexibility market. The work is described in detail in [1].[1] Operationally, this optimization model can be a tool for DSOs to calculate the flexibility requests, to avoid or mitigate network congestions for a specific time period and under a particular load profile in that network. Therefore, this request will be sent to the aggregator, the entity responsible for providing a service to the network operator and activating the flexibility based on that request. The optimization problem is developed to minimize the aggregator operation costs. The costs are based on curtailing local generation output, charging or discharging batteries, switching of curtailable and disconnectable loads and shifting loads during specific time periods.

Mathematical Formulation

The mathematical formulation that follows is meant to work as a guide to go through the code and better understand it. The same model is coded in two different versions: one written with the python *gurobi* API, and the other written in *pyomo*. No distinction is made in the models for loads and generators, and they are going to be simply called "assets" from now on. The objective function is to minimize the total flexibility costs for activating both active and reactive power.

[1] Many citations of the journal article are used in the following handout summary.

First simple conceptual model

This first model formulation should help the reader to better understand conceptually how the model works and what they do. The model is written in a relatively simple language. Mind that the variables, sets and constraints (equations) represented here does not picture exactly the written code.[2] This conceptual model is built with the assumption the every node with an active or reactive asset defined, can provide flexibility. Effectively, this means that a variable representing flexibility is declared at each network node.

$$\underset{\Xi^{\text{AC OPF}}}{\text{Minimize}} \sum_{n}^{N} C_n^{P^{up}} flex_n^{P^{up}} + C_n^{P^{dn}} flex_n^{P^{dn}} + C_n^{Q^{up}} flex_n^{Q^{up}} + C_n^{Q^{dn}} flex_n^{Q^{dn}}$$

subjected to:

$$\begin{aligned}
& P_n = P_{0_n} + flex_n^{P^{up}} - flex_n^{P^{dn}} && \forall n \in \mathcal{N} \\
& Q_n = Q_{0_n} + flex_n^{Q^{up}} - flex_n^{Q^{dn}} && \forall n \in \mathcal{N} \\
& P_n = P_n^{in} - P_n^{out} && \forall n \in \mathcal{N} \\
& Q_n = Q_n^{in} - Q_n^{out} && \forall n \in \mathcal{N} \\
& V_n^{min} \leq V_n \leq V_n^{max}, && \forall n \in \mathcal{N} \\
& V_n = 1 && \forall n \in \mathcal{N}_{slack} \\
- & S_{ij}^{max} \leq S_{ij} \leq S_{ij}^{max} && \forall ij \in \Lambda \\
& P_{slack} = P_n^{in} - P_n^{out} && \forall n \in \mathcal{N}_{slack} \\
& Q_{slack} = Q_n^{in} - Q_n^{out} && \forall n \in \mathcal{N}_{slack} \\
& V_i - V_j = \frac{P_{ij} R_{ij} + Q_{ij} X_{ij}}{V_{base}^2} && \forall ij \in \Lambda \\
& 0 \leq flex_n^{P^{up}} \leq flex_{max_n}^{P^{up}} && \forall n \in \mathcal{N} \\
& 0 \leq flex_n^{P^{dn}} \leq flex_{max_n}^{P^{dn}} && \forall n \in \mathcal{N} \\
& 0 \leq flex_n^{Q^{up}} \leq flex_{max_n}^{Q^{up}} && \forall n \in \mathcal{N} \\
& 0 \leq flex_n^{Q^{dn}} \leq flex_{max_n}^{Q^{dn}} && \forall n \in \mathcal{N}
\end{aligned}$$

[2] For an exact representation of the model coding, have a look at the next sub-chapters.

Table 1: List of symbols: Conceptual Model

Symbol	Type	Description
n	Index	Index for network nodes
i	Index	Index for general "From-Node" in the line
j	Index	Index for general "To-Node" in the line
$slack$	Index	Index for slack nodes
$\mathcal{N}$	Set	Set of network nodes
$\mathcal{N}_{slack}$	Set	Set of network slack nodes
Λ	Set	Set of power lines
V_{base}	Parameter	Voltage base
V_n^{max}	Parameter	Max Voltage at node n
V_n^{min}	Parameter	Min Voltage at node n
C_n	Parameter	Marginal flexibility cost at node n
R_{ij}	Parameter	Absolute value of the resistance of line (i, j)
X_{ij}	Parameter	Absolute value of the reactance of line (i, j)
${flex^P_{max_n}}^{up}$	Parameter	Maximum increase of active power at node n
${flex^Q_{max_n}}^{up}$	Parameter	Maximum increase of reactive power at node n
${flex^P_{max_n}}^{dn}$	Parameter	Maximum decrease of active power at node n
${flex^Q_{max_n}}^{dn}$	Parameter	Maximum decrease of reactive power at node n
V_n	Continuous variable	Voltage magnitude at node n
S_{ij}	Continuous variable	Apparent power flow over power line (i, j)
P_{ij}	Continuous variable	Active power flow over power line (i, j)
Q_{ij}	Continuous variable	Reactive power flow over power line (i, j)
P_n	Continuous variable	Active power injection in node n
Q_n	Continuous variable	Reactive power injection in node n
$flex_n^{P^{up}}$	Continuous variable	Active power flexibility request to increase consumption/production in node n
$flex_n^{P^{dn}}$	Continuous variable	Active power flexibility request to decrease consumption/production in node n
$flex_n^{Q^{up}}$	Continuous variable	Reactive power flexibility request to increase consumption/production in node n
$flex_n^{Q^{dn}}$	Continuous variable	Reactive power flexibility request to decrease consumption/production in node n

Pyomo model formulation in detail

This first in-depth formulation represents better the AC Flex OPF written in *pyomo*. Here, the same approach as above is used: every node can theoretically provide flexibility, meaning that flex variables are defined for each node. The model accepts as inputs some "starting powers", which defines the initial loads or generators size. Variables, sets and constraints shown below do are exactly as coded in the pyomo model.

$$\underset{\Xi^{\text{AC OPF}}}{\text{Minimize}} \sum_{n}^{N} C_n^{P^{up}} flex_n^{P^{up}} + C_n^{P^{dn}} flex_n^{P^{dn}} + C_n^{Q^{up}} flex_n^{Q^{up}} + C_n^{Q^{dn}} flex_n^{Q^{dn}}$$

subjected to:

$$\begin{aligned}
&\sum_i \mathcal{M}_n^{in} P_{ij} - \sum_j \mathcal{M}_n^{out} P_{ij} = P_{0n} + flex_n^{P^{up}} - flex_n^{P^{dn}} && \forall n \in \mathcal{N} \\
&\sum_i \mathcal{M}_n^{in} Q_{ij} - \sum_j \mathcal{M}_n^{out} Q_{ij} = Q_{0n} + flex_n^{Q^{up}} - flex_n^{Q^{dn}} && \forall n \in \mathcal{N} \\
&P_{slack} = \sum_i \mathcal{M}_n^{in} P_{ij} - \sum_j \mathcal{M}_n^{out} P_{ij} && \forall n \in \mathcal{N}_{slack} \\
&Q_{slack} = \sum_i \mathcal{M}_n^{in} Q_{ij} - \sum_j \mathcal{M}_n^{out} Q_{ij} && \forall n \in \mathcal{N}_{slack} \\
&E_n^{min} \leq E_n \leq E_n^{max}, && \forall n \in \mathcal{N}_{all} \\
&E_n = 1 && \forall n \in \mathcal{N}_{slack} \\
&E_i - E_j = \frac{P_{ij} R_{ij} + Q_{ij} X_{ij}}{V_{base}^2} && \forall ij \in \Lambda \\
&0 \leq flex_n^{P^{up}} \leq flex_{max_n}^{P^{up}} && \forall n \in \mathcal{N} \\
&0 \leq flex_n^{P^{dn}} \leq flex_{max_n}^{P^{dn}} && \forall n \in \mathcal{N} \\
&0 \leq flex_n^{Q^{up}} \leq flex_{max_n}^{Q^{up}} && \forall n \in \mathcal{N} \\
&0 \leq flex_n^{Q^{dn}} \leq flex_{max_n}^{Q^{dn}} && \forall n \in \mathcal{N} \\
-&S_{ij}^{max} \leq P_{ij} + A_q Q_{ij} \leq S_{ij}^{max}, && \forall ij \in \Lambda \\
-&S_{ij}^{max} \leq P_{ij} - A_q Q_{ij} \leq S_{ij}^{max}, && \forall ij \in \Lambda \\
-&S_{ij}^{max} \leq P_{ij} A_q + Q_{ij} \leq S_{ij}^{max}, && \forall ij \in \Lambda \\
-&S_{ij}^{max} \leq P_{ij} A_q - Q_{ij} \leq S_{ij}^{max}, && \forall ij \in \Lambda
\end{aligned}$$

where $\Xi^{\text{AC OPF}} = \{E_n, P_{ij}, Q_{ij}, P_{slack}, Q_{slack}, flex_n^{P^{up}}, flex_n^{P^{dn}}, flex_n^{Q^{up}}, flex_n^{Q^{dn}}\}$ is the set of optimization variables.

Table 2: List of symbols: Pyomo AC Flex OPF Model

Symbol	Type	Description
n	Index	Index for network nodes
i	Index	Index for general "From-Node" in the line
j	Index	Index for general "To-Node" in the line
$slack$	Index	Index for slack nodes
$\mathcal{N}$	Set	Set of network nodes without slacks
$\mathcal{N}_{slack}$	Set	Set of network slack nodes
$\mathcal{N}_{all}$	Set	Set of all network nodes
Λ	Set	Set of power lines
$\mathcal{M}_n^{in}$	Set	Set of "Nodes-In" at node n (the i bus of a (i,j) line)
$\mathcal{M}_n^{out}$	Set	Set of "Nodes-Out" at node n (the j bus of a (i,j) line)
A_q	Parameter	Derivative of the line that form a convex polygon $1/\sqrt(2)$
V_{base}	Parameter	Voltage base
E_n^{max}	Parameter	Max Voltage at node n
E_n^{min}	Parameter	Min Voltage at node n
S_{ij}^{max}	Parameter	Max apparent power flowing in line (i,j)
C_n	Parameter	Marginal flexibility cost at node n
R_{ij}	Parameter	Absolute value of the resistance of line (i,j)
X_{ij}	Parameter	Absolute value of the reactance of line (i,j)
${flex^P_{max_n}}^{up}$	Parameter	Maximum increase of active power at node n
${flex^Q_{max_n}}^{up}$	Parameter	Maximum increase of reactive power at node n
${flex^P_{max_n}}^{dn}$	Parameter	Maximum decrease of active power at node n
${flex^Q_{max_n}}^{dn}$	Parameter	Maximum decrease of reactive power at node n
E_n	Continuous variable	Voltage magnitude at node n
P_{ij}	Continuous variable	Active power flow over power line (i,j)
Q_{ij}	Continuous variable	Reactive power flow over power line (i,j)
P_{slack}	Continuous variable	Active power injection from grid at slack n
Q_{slack}	Continuous variable	Reactive power injection in node n
$flex_n^{P^{up}}$	Continuous variable	Active power flexibility request to increase consumption/production in node n
$flex_n^{P^{dn}}$	Continuous variable	Active power flexibility request to decrease consumption/production in node n
$flex_n^{Q^{up}}$	Continuous variable	Reactive power flexibility request to increase consumption/production in node n
$flex_n^{Q^{dn}}$	Continuous variable	Reactive power flexibility request to decrease consumption/production in node n

Gurobi model formulation in detail

The formulation in the gurobi model is substantially the same as the previous one in the pyomo model. Difference lies in the programming style, in particular regarding the definition of an additional model set $\mathcal{A}$ which contains all model assets capable of flexibility and eventually the definition of set $\mathcal{A}_n$ at each network node n which contains the list of assets present in the node. This allows to define flex variables only for nodes where an asset is assigned, turning into a more efficient model when the size will grow exponentially. Many code structures are built on the set of lines Λ which contains ℓ elements such that $\ell = (i, j)$, with i the "fromNode" and j the "toNode". Changes in nomenclature are also present, following the code formulation.

$$\underset{\Xi^{\text{AC OPF}}}{\text{Minimize}} \sum_{i}^{N} C_i^{P^{up}} flex_i^{P^{up}} + C_i^{P^{dn}} flex_i^{P^{dn}} + C_i^{Q^{up}} flex_i^{Q^{up}} + C_i^{Q^{dn}} flex_i^{Q^{dn}}$$

subjected to:

$$\begin{aligned}
&\sum_{\ell} \mathcal{M}_n^{in} P_\ell - \sum_{\ell} \mathcal{M}_n^{out} P_\ell = P_{0_n} + \sum_{i} \mathcal{A}_n flex_i^{P^{up}} - \sum_{i} \mathcal{A}_n flex_i^{P^{dn}} && \forall n \in \mathcal{N} \\
&\sum_{\ell} \mathcal{M}_n^{in} Q_\ell - \sum_{\ell} \mathcal{M}_n^{out} Q_\ell = Q_{0_n} + flex_i^{Q^{up}} - flex_i^{Q^{dn}} && \forall n \in \mathcal{N} \\
&P_{slack} = \sum_{\ell} \mathcal{M}_n^{in} P_\ell - \sum_{\ell} \mathcal{M}_n^{out} P_\ell && \forall n \in \mathcal{N}_{slack} \\
&Q_{slack} = \sum_{\ell} \mathcal{M}_n^{in} Q_\ell - \sum_{\ell} \mathcal{M}_n^{out} Q_\ell && \forall n \in \mathcal{N}_{slack} \\
&E_n^{min} \leq E_n \leq E_n^{max}, && \forall n \in \mathcal{N}_{all} \\
&E_n = 1 && \forall n \in \mathcal{N}_{slack} \\
&E_i - E_j = \frac{P_\ell R_\ell + Q_\ell X_\ell}{V_{base}^2} && \forall \ell = (i, j) \in \Lambda \\
&0 \leq flex_i^{P^{up}} \leq flex_{max_i}^{P^{up}} && \forall i \in \mathcal{A} \\
&0 \leq flex_i^{P^{dn}} \leq flex_{max_i}^{P^{dn}} && \forall i \in \mathcal{A} \\
&0 \leq flex_i^{Q^{up}} \leq flex_{max_i}^{Q^{up}} && \forall i \in \mathcal{A} \\
&0 \leq flex_i^{Q^{dn}} \leq flex_{max_i}^{Q^{dn}} && \forall i \in \mathcal{A} \\
&-S_\ell^{max} \leq P_\ell + A_q Q_\ell \leq S_\ell^{max}, && \forall \ell \in \Lambda \\
&-S_\ell^{max} \leq P_\ell - A_q Q_\ell \leq S_\ell^{max}, && \forall \ell \in \Lambda \\
&-S_\ell^{max} \leq P_\ell A_q + Q_\ell \leq S_\ell^{max}, && \forall \ell \in \Lambda \\
&-S_\ell^{max} \leq P_\ell A_q - Q_\ell \leq S_\ell^{max}, && \forall \ell \in \Lambda
\end{aligned}$$

where $\Xi^{\text{AC OPF}} = \{E_n, P_\ell, Q_\ell, P_{slack}, Q_{slack}, flex_i^{P^{up}}, flex_i^{P^{dn}}, flex_i^{Q^{up}}, flex_i^{Q^{dn}}\}$ is the set of optimization variables.

In the table below the difference in the symbols are summarized.

Table 3: List of symbols: Gurobi AC Flex OPF Model

Symbol	Type	Description
ℓ	Index	Index for power lines
i	Index	Index for network assets
(i, j)	Index	Index for (FromNode,ToNode) of power line ℓ
$\mathcal{A}$	Set	Set of network assets
$\mathcal{A}_n$	Set	Set of network assets at node n
C_i	Parameter	Marginal flexibility cost of unit i

References

[1] P. Olivella-Rosell, P. Lloret-Gallego, Í. Munné-Collado, R. Villafafila-Robles, A. Sumper, S. Ø. Ottessen, J. Rajasekharan, and B. A. Bremdal, "Local flexibility market design for aggregators providing multiple flexibility services at distribution network level," *Energies*, vol. 11, no. 4, p. 822, 2018.